



zk-Database: Privacy-enabled Databases using Zero-Knowledge Proof

Gholamreza Ramezan
PlayFi Labs
Vancouver, Canada
gramezan@ece.ubc.ca

Ben Beath
PlayFi Labs
New York, USA
ben@liftdata.ai

Eladio Robles Casas
PlayFi Labs
Santa Cruz de Tenerife, Spain
eladio@liftdata.ai

Jake Godfrey
PlayFi Labs
Melbourne, Australia
jake@liftdata.ai

Abstract

In today's digital landscape, there is an increasing demand for scalable solutions that can handle vast amounts of data while ensuring robust security and privacy. A challenge arises when offloading data storage and query tasks to powerful yet untrusted servers in cloud computing and decentralized systems. To address this issue, we propose zk-Database, which leverages zero-knowledge proof (ZKP) and blockchain to guarantee the integrity and privacy of data. zk-Database represents a significant advancement in secure data management and offers a framework where computational tasks can be outsourced without compromising trustworthiness. We investigate various commitment schemes, including Merkle trees and Pedersen commitments. We apply ZKP methods such as Halo2, Risc Zero, and Lasso&Jolt to prove the data storage and query operations in the database. Our performance evaluation indicates that database queries are executed in 11 ms with Halo2, which outperforms Risc Zero and Lasso&Jolt, requiring 17.56 ms and 158 ms, respectively. The performance difference is primarily due to Risc Zero's and Lasso&Jolt's use of a virtual machine approach, which translates programs into RISC-V before generating a ZKP. Although Risc Zero and Lasso&Jolt are less efficient for individual query operations, they offer greater capability in handling complex database queries. Specifically, Lasso&Jolt performs database store operations approximately six times faster than Risc Zero. However, Risc Zero's write operation is 3.5 times slower compared to Halo2.

CCS Concepts

• Security and privacy → Cryptography; Database and storage security.

Keywords

blockchain, database, zero-knowledge proof

ACM Reference Format:

Gholamreza Ramezan, Eladio Robles Casas, Ben Beath, and Jake Godfrey. 2024. zk-Database: Privacy-enabled Databases using Zero-Knowledge Proof. In *2024 7th International Conference on Blockchain Technology and Applications (ICBTA) (ICBTA 2024)*, December 06–08, 2024, Xi'an, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3708622.3708634>

1 Introduction

In today's digital landscape, the need for scalable solutions capable of managing extensive data while ensuring robust security and privacy is increasingly critical. One of the key challenges in this realm involves offloading data storage and query tasks to powerful yet potentially untrusted servers, a scenario prevalent in cloud computing and distributed systems. While leveraging the computational power of remote servers offers significant advantages, it also raises concerns regarding the integrity and confidentiality of the processed data.

Privacy-enabled databases have been developed to tackle the challenge of securely managing and querying data in environments where servers are both powerful and potentially untrusted. These databases are designed to not only protect data privacy but also to ensure the accuracy and reliability of query results. Furthermore, privacy-enabled databases often incorporate mechanisms for secure data sharing and access control, which bolsters their resilience against potential threats and ensures compliance with rigorous privacy regulations.

The need for both data integrity and privacy is pressing across various domains. For example, in the healthcare sector, regulatory bodies such as the Food and Drug Administration (FDA) require accurate patient data to ensure drug safety while complying with privacy laws like the Health Insurance Portability and Accountability Act (HIPAA) [13, 14]. Financial institutions, too, are subject to audits by the Securities and Exchange Commission (SEC), where privacy-preserving databases could facilitate the verification of financial statements without exposing sensitive information [15]. In democratic elections, election commissions need to demonstrate vote accuracy and process integrity while safeguarding individual voter choices. Additionally, in supply chain management, privacy-preserving databases can validate product authenticity and ethical sourcing without disclosing proprietary information. Another example is the U.S. Department of Education (DoE) that collects students outcome statistics for their College Scorecards. Ensuring accuracy while maintaining confidentiality is a significant challenge

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICBTA 2024, Xi'an, China

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1739-0/24/12
<https://doi.org/10.1145/3708622.3708634>

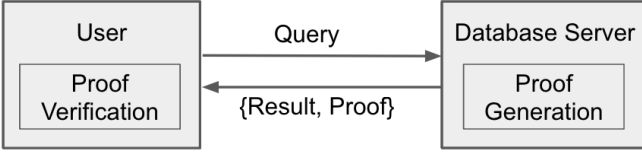


Figure 1: Main data flow in zk-Database.

for DoE. The Columbia University’s 2022 ranking drop due to incorrect data submissions illustrates how existing database systems are struggling with verifying data [10]. ZKP technology promises to enhance privacy while ensuring the secure storage and retrieval of sensitive information.

A notable application of ZKP technology is in decentralized database query services, where the database server provides query results along with proof. This setup enables users to verify the correctness of query results against a digest before executing the service transaction.

In this paper, we present a practical scheme that ensures privacy and accurately performs result verification for arbitrary database queries using blockchain systems. We leverage ZKPs [1, 11, 12] and propose zk-Database which guarantees the validity of data being stored (i.e, database write operation) and verifies the accuracy of query results in databases. We also utilize the Merkle Tree Vector commitment method to ensure the integrity of data records. Fig. 1 illustrates the components and workflow involved in querying a database with an integrated ZKP module.

The zk-Database method involves a database of values shared between two parties: the prover and the verifier. The prover stores a value in the database and commits to it. Later, the prover can read this value and make a claim about the previously committed data. This scheme guarantees the following properties. 1) Hiding property: Values stored in the database are hidden from a verifier. 2) Binding property: Once a value is committed to a specific position in the database, the prover will not read a different value. 3) ZKPs used for reading or writing a value ensure that the value remains concealed while guaranteeing that the query execution for retrieving data is performed correctly. Our proposed scheme uses different ZKP mechanisms including Halo2 and Lasso&Jolt. The architecture of zk-Databases allows the proof generation process to be distributed across multiple untrusted computers, effectively balancing the load and ensuring that application performance requirements are met.

zk-Database is a reliable and practical solution for verifying arbitrary queries in blockchain-based systems. Our key contributions are summarized below:

- We propose a scheme to integrate vector commitments and ZKP methods into the database servers.
- We implement the proposed scheme utilizing the Halo2, Risc Zero, and Lasso&Jolt ZKP mechanisms as well.
- We also evaluate the performance of our method in comparison with other vector commitment techniques, including Pedersen and Updatable Vector Commitment schemes, as well as other ZKP methods such as Halo2, Risc Zero and

Lasso&Jolt. The results demonstrate that Halo2 exhibits superior performance in both query and write operations, completing them in 11 ms and 632 ms, respectively. In comparison, Risc Zero takes 17 ms and 13 s, while Lasso&Jolt requires 158 ms and 2 s for the same operations. Despite Halo2’s faster performance, Risc Zero and Lasso&Jolt offer greater flexibility in designing ZKP circuits for complex queries.

The structure of this paper is as follows: Section 2 discusses related works, while Section 3 covers the fundamental concepts. Section 4 provides an in-depth look at the main components, data flow, and adversary model of our method. Section 5 assesses the performance through experimental analysis, and Section 6 concludes with final remarks.

2 Related Work

Recent research has explored integrating database technologies with ZKP [3, 5, 6, 8, 16, 17]. These works can be categorized into two main areas: elementary databases, which use commitment schemes to ensure the existence of data values within a database, and complex-query databases, which use ZKP methods to verify the correctness of complex queries, such as those involving joins between multiple datasets. ZKP enables a prover to convince a verifier that a statement is true without disclosing any additional information beyond the statement’s validity. Applied to databases, ZKP facilitates the secure execution of queries on untrusted servers while ensuring that the results are accurate and unaltered. This approach is crucial for scenarios where data privacy and the integrity of query outcomes are essential, providing a mechanism for secure database interactions even on potentially adversarial platforms. The ZKP-enabled database represents a significant advancement in secure data management that offers a framework where computational tasks can be outsourced without compromising trustworthiness.

Commitment schemes are designed to allow one party to commit to a value while keeping it hidden, with the option for later disclosure and verification. They are typically used to ensure that a party cannot alter the value once it is stored in the database and that the value is revealed only at a later time. The authors in [6] and [17] utilized the mercurial commitment schemes with Merkle trees. In [6], the authors introduced zero-knowledge elementary databases (ZK-EDBs), which generalize zero-knowledge sets to elementary databases (EDBs). This method verifies the membership of data in a database without disclosing the database’s size. Dummy commitments allow the sender to commit to the database D in polynomial time, while the actual size of the database is concealed by imposing a super-polynomial upper bound on the number of leaves in the Merkle tree. Each leaf is assigned a key x and contains a commitment to the value $y = D(x)$, with internal nodes containing commitments to their child nodes. This method falls into the first category. In [17], the authors introduced zero-knowledge functional elementary database (ZK-FEDB), which extends ZK-EDB to support general functional queries that operate on multiple datasets. Specifically, ZK-FEDB allows the prover to respond to queries of the form "send me all records (x, v) in D satisfying $f(x, v) = 1$ ", where f is any Boolean circuit. ZK-FEDB uses zero-knowledge succinct

Table 1: Privacy Schemes in Databases

Database Scheme	Commitment and ZKP Scheme
ZK-EDB [6]	Mercurial Merkle-based
ZK-FEDB [17]	Mercurial Merkle-based, zk-SNARK
Updatable Database [3, 8]	Updatable VC
zkQuery [16]	Polynomial (e.g. Pedersen), Halo

non-interactive arguments of knowledge (zk-SNARKs) and falls into the second category.

Works in [8] and [3] focus on non-Merkle-based vector commitment approaches. The proposed method verifies whether a given data record in the database has been updated and committed to the updated database. The work in [8] allows the prover to efficiently demonstrate in zero-knowledge that a value is stored in the database at a certain position. The work in [3] provided privacy mechanisms to read, write and update operations. It enables the prover to efficiently conceal the read and write positions from the verifier. Hence, a prover can read, write, and update values in a database and prove to a verifier in zero-knowledge that a value is read from or written into a certain position. Both methods belong to the first category.

The authors in [5] and [16] offer more robust methods for constructing ZKP for SQL-based queries. The work in [5] presents a proof method for set-based queries called zkSQL. Set-based proofs verify properties of query results, such as confirming that the output of a sort operation retains the same set of tuples as the input and that the output is monotonically increasing. The authors in [16] introduced zkQuery, a method for SQL-based queries in relational databases for blockchains. zkQuery contains three main entities: the client, the storage service providers, and the validators within the blockchain. The outsourced database of clients is organized in a matrix A . The client utilizes the Enc function to encrypt its database by using searchable encryption scheme F such that $F(A; k) = \{F(A_{ij}; k)\}$, where k is the secret key. The encrypted database is later sent to the contracted service provider. The encrypted matrix is preserved by the storage service provider, which views the matrix as a function. A corresponding commitment is uploaded to the blockchain. Later, the validators can verify the validity of the query results with the help of the commitment. A polynomial commitment scheme, e.g., Pedersen, is used. We summarize the aforementioned related works in Table 1.

3 Fundamentals

3.1 Commitment Scheme

A commitment scheme hides the committed value x because, given the commitment C , it is computationally infeasible to determine x without knowing a random value r . The randomness introduced by r ensures that different values of x map to different commitments even if x has not changed. This hides x from anyone who sees C alone. Also, $\mathbb{G}$ is a group of prime order p . We now review different commitment schemes.

3.1.1 Pedersen Commitment [7]. A pair of probabilistic polynomial time algorithms (Setup, Com) comprise a non-interactive commitment scheme which are defined as follows [6].

- $pp \leftarrow \text{Setup}(1^\lambda)$: given the security parameter λ , this function outputs public parameters pp .
- $C \leftarrow \text{Com}(x; r)$: it outputs the commitment to the value x in the value set X considering a randomness r in the randomness space R ,

Let $x, r \in \mathbb{F}_p$, $\text{Setup}: pp = (g, h) \xleftarrow{\$} \mathbb{G}$, $\text{Com}(x; r) = g^x h^r$.

3.1.2 Pedersen Polynomial Commitment. Given a polynomial $p(x)$ with a degree less than d and coefficients a , e.g.,

$$p(x) = \sum_{i=0}^{d-1} a_i x^i,$$

$\text{Setup}: G \xleftarrow{\$} \mathbb{G}^n, h \xleftarrow{\$} \mathbb{G}, pp = (G, h), \text{Com}(p(x); r) = \langle a, G \rangle \cdot h^r$

3.1.3 Kate Polynomial Commitment [4]. Given a polynomial $p(x)$ with a degree less than d and coefficients a , e.g.,

$$p(x) = \sum_{i=0}^{d-1} a_i x^i,$$

$\text{Setup}: \alpha \xleftarrow{\$} \mathbb{F}_p, g \xleftarrow{\text{generator}} \mathbb{G}, pp = (g, g^\alpha, g^{\alpha^2}, \dots, g^{\alpha^{d-1}}),$
 $\text{Com}(p(x)) = \prod_{i=0}^{d-1} (g^{\alpha^i})^{a_i}.$

3.1.4 Updatable Vector Commitment [3, 8].

- $\text{Setup}(1^k, l)$: This function outputs $pp = (p, \mathbb{G}, \tilde{\mathbb{G}}, \mathbb{G}_t, e, g, \tilde{g}, g_1, \tilde{g}_1, \dots, g_l, \tilde{g}_l, g_{l+1}, \dots, g_{2l})$, where l is the maximum number of data entries, and $\mathbb{G}, \tilde{\mathbb{G}}$, and $\mathbb{G}_t$ are groups of prime order p . The element g is a generator of $\mathbb{G}$, and $\tilde{g}$ is a generator of $\tilde{\mathbb{G}}$. The mapping $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \mathbb{G}_t$ satisfies bilinearity, i.e., $e(g^x, \tilde{g}^y) = e(g, \tilde{g})^{xy}$, and non-degeneracy, i.e., for all generators $g \in \mathbb{G}$ and $\tilde{g} \in \tilde{\mathbb{G}}$, $e(g, \tilde{g})$ generates $\mathbb{G}_t$. Additionally,

$$g_i = g^{\alpha^i}$$

$$\tilde{g}_i = \tilde{g}^{\alpha^i}$$

where α is a random element of $\mathbb{Z}_p$.

- $\text{Commitment}(X; r)$: This function outputs

$$vc = g^r \prod_{j=1}^n g_{l+1-j}^{X[j]} = g^r g_l^{X[1]} \dots g_{l+1-n}^{X[n]}$$

where r is a random value.

- $\text{Witness}(i, X, r)$: This function outputs

$$w_i = g_i^r \prod_{j=1, j \neq i}^n g_{l+1-j+i}^{X[j]}$$

for $X[i]$.

- $\text{Verification}(vc, x, i, w)$: This function outputs 1 if w is a valid witness for value x at position i given a vector X with commitment vc , and 0 otherwise. The verification is done by checking the following equation:

$$e(vc, \tilde{g}_i) = e(w, \tilde{g}) \cdot e(g_l, \tilde{g}_l)^x.$$

This scheme has a capability for partial update when updating value x with a new value x' .

- *Commitment Update*(vc, j, x, r, x', r'): After updating a new value x' to the database at position j , this function updates the commitment vc to vc' if value x at position j and randomness r changes to value x' and randomness r' . The values in other positions remain unchanged. The updated commitment is given by

$$vc' = vc \cdot g^{r'-r} \cdot g_{l+1-j}^{x'-x}$$

- *Witness Update*(w, i, j, x, r, x', r'): This function updates witness w for position i when value x at position j and randomness r change to value x' and randomness r' . If $i = j$, it outputs w ; otherwise, it outputs the updated witness

$$w' = w \cdot g_i^{r'-r} \cdot g_{l+1-j+i}^{x'-x}$$

- *Commitment Update Verification*($vc, vc', w, j, x, r, x', r'$): This function first computes $v = \text{Verification}(vc, x, j, w)$. If $v = 0$, it outputs 0. Otherwise, it runs $\bar{vc} = \text{Commitment Update}(vc, j, x, r, x', r')$, and if $\bar{vc} = vc'$, it outputs 1; otherwise, it outputs 0.

3.1.5 Mercurial Merkle Tree Vector Commitment [6]. Each internal node holds a commitment to its two child nodes. Dummy commitments are crucial because they allow the sender to commit to the database in polynomial time while keeping its size confidential. This confidentiality is maintained by using a super-polynomial upper bound on the number of leaves in the Merkle tree. Each leaf is assigned a value x and includes a genuine commitment to the corresponding value $y = D(x)$. If $D(x) \neq \text{nil}$ (i.e., $x \in D$) then $C_x = \text{Hash}(D(x), r_x)$. If $D(x) = \text{nil}$ and its sibling has a value in D , then $C_x = \text{Hash}(r_x)$. Otherwise, $C_x = \text{nil}$. Additionally, every internal node contains a commitment to its two children. By incorporating a dummy commitment at the root of each empty subtree, the sender can efficiently commit to the entire set $D = \{(x, y)\}$ in polynomial time.

The Merkle tree utilizes a collision-resistant hash function to create a vector commitment. In this structure, the hashes of the database values form the leaves of a tree with height $h = \lceil \log_c(N) \rceil$, where c denotes the number of children of each node (e.g., for binary Merkle trees $c = 2$) and N is the size of the database. Each non-leaf node's hash is computed by hashing its child nodes. The root node is at depth 0, and the depth of the leaves is $\lceil \log_c(N) \rceil$. The *Com* function outputs the Merkle root C as the commitment. For proving the inclusion of a value x , the proof consists of $h(c - 1)$ hashes, representing the siblings of the nodes on the path from the root to the hash of x .

3.2 ZKP

In this subsection, we will review well-known ZKP protocols and frameworks.

3.2.1 Halo2. Halo is the first practical example of recursive proof composition without a trusted setup based on zk-SNARK proof system. All previously known realizations of recursive proof composition have required a trusted setup and cycles of expensive

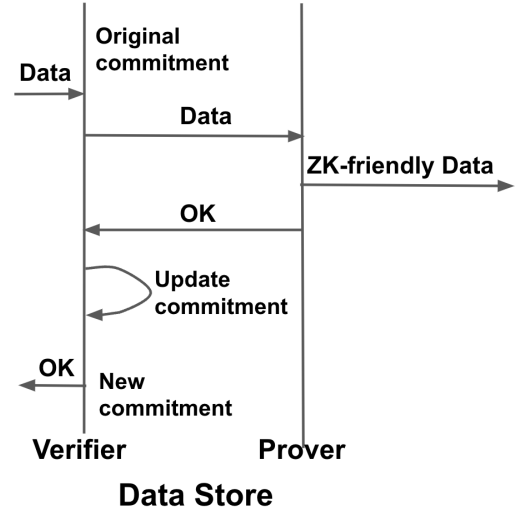


Figure 2: Data store workflow.

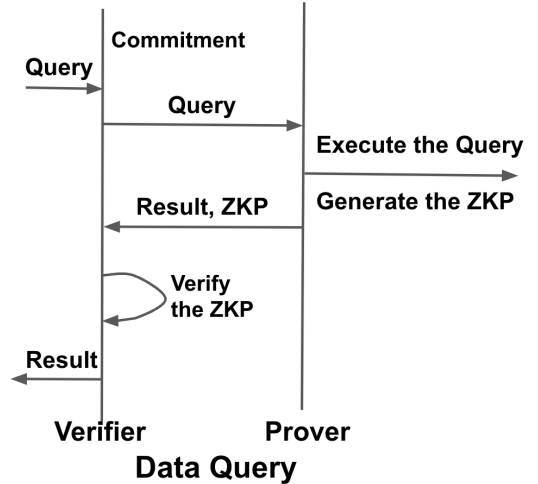


Figure 3: Data query workflow.

pairing-friendly elliptic curves. Halo2 is like Halo but with a different arithmetization. It utilizes PLONK method instead of Sonic in Halo. Halo2 is a popular ZKP system implemented in Rust [2].

3.2.2 Lasso&Jolt. Lasso, a novel family of lookup arguments, significantly enhances zk-SNARK performance. It allows an untrusted prover to commit to a vector $a \in \mathbb{F}^m$ and prove that all entries of a reside in a predetermined table $t \in \mathbb{F}^n$. Lasso's performance characteristics unlock the so-called "lookup singularity" by efficiently managing large tables, such as those with 2^{128} entries. Instead of incurring costs proportional to table size, Lasso maintains costs based on the number of lookups performed, thus avoiding the need to fully materialize the table. This approach, built upon Spark—a time-optimal polynomial commitment scheme for sparse polynomials—enables more scalable and efficient verification.

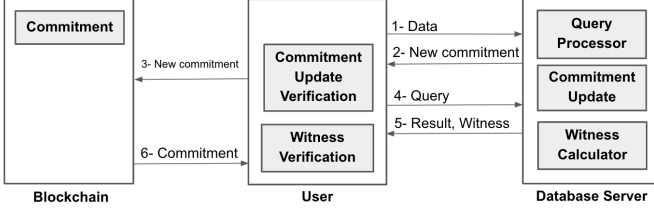


Figure 4: zk-Database Components with Vector Commitment Schemes.

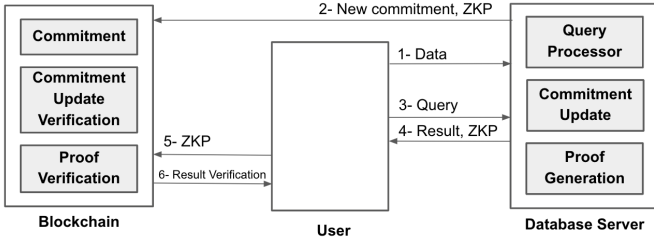


Figure 5: zk-Database Components with ZKP Schemes.

Building on Lasso, Jolt refines and extends these capabilities by streamlining proof generation and verification, further reducing data exchange and processing overhead. It transforms CPU instructions into Lasso lookups, efficiently managing entire instruction sets, such as RISC-V or WebAssembly. By decomposing operations into smaller, manageable lookups, Jolt addresses previous limitations in zero-knowledge virtual machines (zkVMs) related to preprocessing and prover time [1, 11].

3.2.3 Risc Zero. Risc Zero technology integrates advanced cryptographic concepts, notably STARKs (Scalable Transparent Arguments of Knowledge) and SNARKs, to enhance blockchain security and efficiency. STARKs provide scalability and transparency by using hash functions and Merkle trees, requiring no trusted setup and allowing for verification of computations with minimal trust assumptions. In contrast, SNARKs offer succinctness and non-interactivity, relying on a trusted setup to generate compact proofs. By encoding computations into polynomial representations for STARKs and arithmetic circuits for SNARKs, Risc Zero provides a scalable ZKP system to generate succinct proofs [9].

4 zk-Database Scheme

In this section, we provide an overview for the architecture of our proposed zk-Database. zk-Database employs a commitment scheme to ensure data integrity and a ZKP scheme to validate the execution of this commitment scheme. We first describe the workflow with which the system goes from a database query statement to providing the client with an authenticated answer. Then, we will review the main components of the system.

4.1 Workflow

Our workflow has two parts. First, the engine sets up the commitments over which we will evaluate our ZKPs as shown in Figures

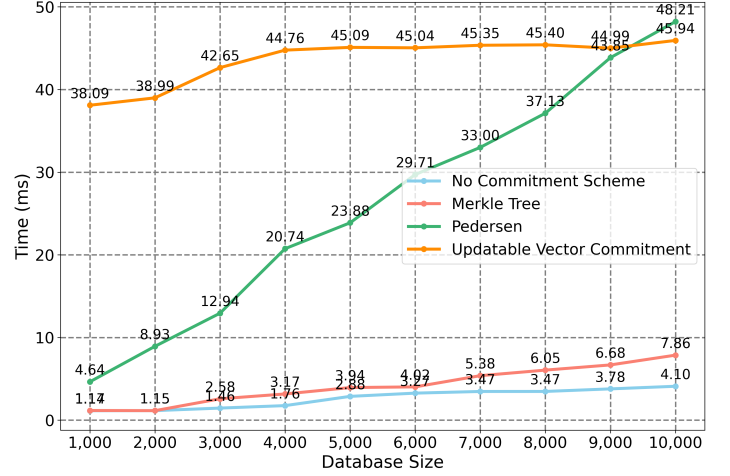


Figure 6: The time of data store operation when using different commitment schemes.

2 and 3. After that, prover and verifier interactively verify the answer to one or more database queries. As the example, a financial institute with its private clients-record database is prover and the US SEC is verifier.

4.2 Main Components

Fig. 4 shows the components of zk-Database with only a vector commitments scheme. The database (i.e., prover) generates a commitment during the database initialization step and sends it to the user (i.e., verifier) to store it on the blockchain for public access. Each time, the user sends a piece of data to store, the database server stores the data and generates a new commitment. The new commitment is then sent to the user, who accepts the data storage if the commitment update verification confirms the new commitment. At a later time, the user sends queries to the database server and the database server sends the result and a witness back to the user. The user will then verify the received data using its witness and existing commitment on the blockchain. If the witness is valid, the user will accept the query result. In this model, the user is trustable as it should run the commitment update verification and witness verification procedures.

Fig. 5 shows the components of zk-Database utilizing ZKP systems. The database generates a commitment during the database initialization step and sends it to the prover. Each time the user sends a piece of data to store, the database server stores the data and generates a new commitment and a proof. The new commitment is then sent to the blockchain, which stores the new commitment if the proof is valid. Then, queries received by the database server will be processed, which generates a result and a proof using the proof generation module and sends it back to the user. The user will then call the proof verification procedure on the blockchain. If the proof is valid, the user will accept the query result. In this model, the user is not trustable as the blockchain will check the new commitment and will verify the proof.

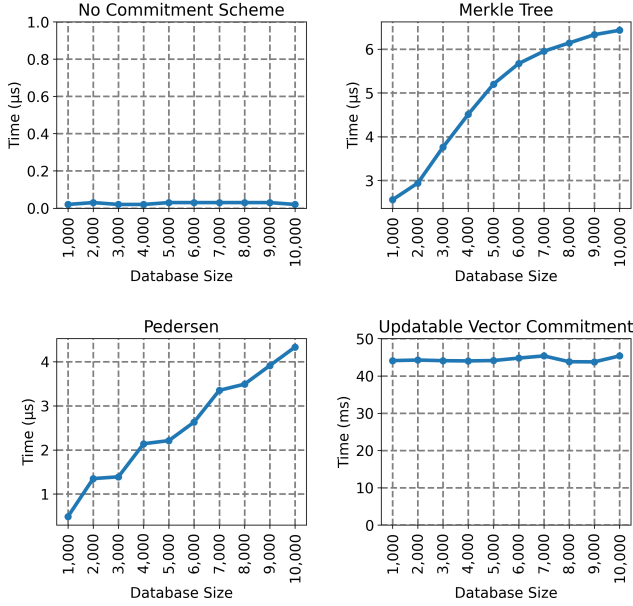


Figure 7: The time of query operation when using different commitment schemes.

5 Performance Evaluation

In this section, we evaluate the performance of zk-Database using different commitment and ZKP schemes. We conduct the evaluation on an Ubuntu 22.04.4 server equipped with an AMD Ryzen 5 3600 6-Core Processor and 64 GB of RAM. The database is built using MongoDB version v4.4.29. We first study the commitment schemes, including Merkle tree, Pedersen, and Improved Updatable VC commitments, as described in Section 3. We also compare the results with a baseline, which is a database operation without applying any commitment scheme. We also study the ZKP frameworks, specifically Halo2, Risc Zero, and Lasso&Jolt. For Halo2 and Lasso&Jolt, we use the implementations provided by the Halo2 and A16Z teams, available at "<https://github.com/zcash/halo2.git>" and "<https://github.com/a16z/jolt>", respectively. The Risc Zero Rust library utilized in this study can be accessed on the "crates.io" website.

Fig. 6 shows the time to write a data record in a database. We vary the size of the database from 1,000 to 10,000 records and obtain the time for different commitment schemes. The basic database operation without utilizing any vector commitment scheme takes less than 5 ms for 10,000 records. The Merkle tree commitment introduces a bit more delay, with a maximum of 7.86 ms for the same number of records, while the Pedersen commitment scheme takes 4.54 ms and increases linearly with database size to 48.21 ms for 10,000 records. However, the Pedersen commitment scheme does not provide a mechanism for partial database updates. Therefore, we consider the Improved Updatable VC scheme, which takes 38.09 ms for 1,000 records but slightly increases with the database size.

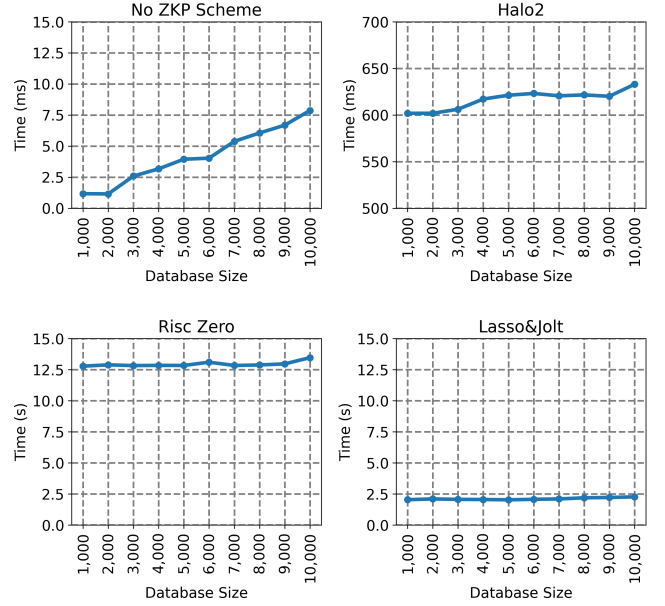


Figure 8: The time of data store operation when using different ZKP frameworks.

Table 2: Time for data storage and query operations using different ZKP frameworks on a 10,000-record database.

ZKP Scheme	Data Store Time (ms)	Data Query Time (ms)
Halo2 [2]	632	11.52
Risc Zero [9]	13,469	17.56
Lasso&Jolt [1]	2,265	158

Fig. 7 shows the time to query the database when using different commitment schemes. As it is shown, the database operation to execute a query and find values based on criteria takes less than 1 μ s. The Pedersen commitment approach takes less than 5 μ s, while the Merkle commitment scheme takes 6.43 μ s. These numbers are much less than the query delay in the Updatable Vector Commitment scheme, which is about 44 ms.

We then consider the Merkle tree commitment scheme for the database and create ZKPs for it using three different methods: Halo2, Risc Zero, and Lasso&Jolt. As shown in Fig. 8, the write operation using Halo2 takes 620 ms and it does not change significantly by increasing the database size. However, the Risc Zero and Lasso&Jolt methods take around 12.5 and 2.1 s, respectively.

As shown in Fig. 9, the database query operation with no ZKP scheme executes around 10 μ s. This delay reaches to around 11, 16, and 155 ms using Halo2, Risc Zero, and Lasso&Jolt, respectively. However, creating a ZKP circuit with Risc Zero and Lasso&Jolt is significantly simpler compared to Halo2. Therefore, if the application can tolerate the longer processing time, we recommend using

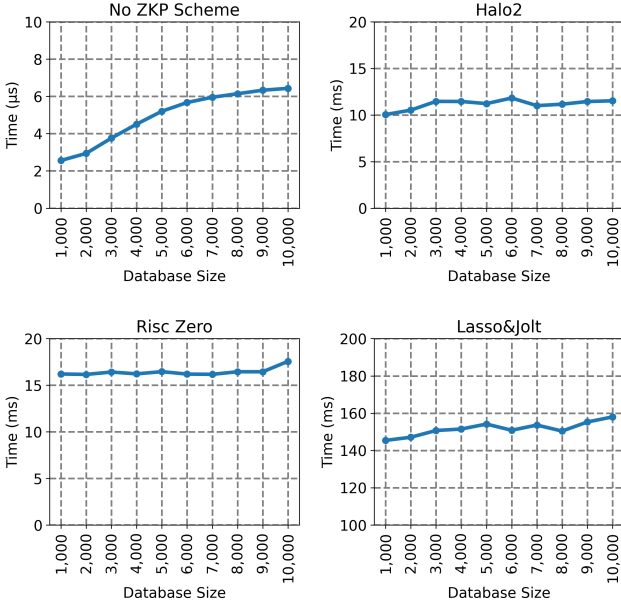


Figure 9: The time of query operation when using different ZKP frameworks.

Risc Zero and Lasso&Jolt. For more complex queries, Risc Zero and Lasso&Jolt will be more efficient in ZKP circuit design.

To provide a comprehensive evaluation of zk-Database performance and facilitate a fair comparison among various ZKP frameworks, we present the results for a database containing 10,000 records in Table 2. The results indicate that Halo2 exhibits the lowest execution time. However, Halo2 poses challenges when designing complex queries. In contrast, Risc Zero and Lasso&Jolt offer enhanced robustness for constructing complex database queries, attributed to their implementation based on a virtual machine (VM) architecture. Nevertheless, the VM architecture introduces additional overhead. Among these, Lasso&Jolt demonstrates superior performance in data store compared to Risc Zero.

To investigate the scalability of the aforementioned methods, we incrementally increased the database size from 10,000 to 100,000 records. In the case of the approach without any ZKP methods, the write and query times only experienced a slight increase of a few microseconds. This pattern remains consistent for the Halo2, Risc Zero, and Lasso&Jolt methods when applied to the larger dataset. The observed write and query times fall within the range of milliseconds, with one thousand times longer than the method without ZKP. Notably, this indicates no significant performance differences as the database size expands to 100,000 records.

6 Conclusion

In this paper, we proposed the zk-Database framework that addresses critical challenges in secure and private data management by leveraging ZKP. Our solution effectively integrates ZKPs into database, ensuring that data privacy and integrity are maintained even when data is handled by potentially untrusted servers.

Our implementation, utilizing the Halo2, Risc Zero, and Lasso&Jolt ZKP frameworks. Halo2 achieves a database store operation of 632 ms which is significantly faster than Risc Zero and Lasso&Jolt that take about 13.4 and 2.2 s, respectively. Also, query execution operations using Risc Zero and Lasso&Jolt are slower, taking about 17.56 and 158 ms, respectively, compared to about 11.52 ms benchmark set by Halo2 approaches. Despite this, zk-Database's design, which incorporates Risc Zero and Lasso&Jolt virtual machines, allows them to handle complex database queries efficiently.

The comparative analysis reveals that while zk-Database introduces additional overhead compared to non-ZKP approaches, it provides substantial gains in data security and privacy. The system's adaptability and robust performance validate its potential for secure, privacy-preserving database management in decentralized computing environments.

In future, we will focus on exploring complex queries to merge different database sets and also enhancing ZKP frameworks to ensure even greater efficiency and security.

References

- [1] Arasu Arun, Srinath Setty, and Justin Thaler. 2024. Jolt: Snarks for Virtual Machines via Lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*. Springer, 3–33.
- [2] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive Proof Composition Without a Trusted Setup. *Cryptology ePrint Archive* (2019). <https://eprint.iacr.org/2019/1021.pdf>
- [3] Jan Camenisch, Maria Dubovitskaya, and Alfredo Rial. 2021. Concise UC Zero-Knowledge Proofs for Oblivious Updatable Databases. In *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*. IEEE, Virtual Conference, 1–16.
- [4] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology - ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security*. Springer, Singapore, 177–194.
- [5] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZkSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023).
- [6] Benoît Libert, Khoa Nguyen, Benjamin Hong Meng Tan, and Huaxiong Wang. 2019. Zero-Knowledge Elementary Databases with More Expressive Queries. In *IACR International Workshop on Public Key Cryptography*. Springer, 255–285.
- [7] Torben Prids Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Annual International Cryptology Conference (CRYPTO)*. Springer, 129–140.
- [8] Alfredo Rial. 2019. UC Updatable Non-Hiding Committed Database with Efficient Zero-Knowledge Proofs. *Cryptology ePrint Archive* (2019).
- [9] Risc0. 2024. *Proof System in Detail*. Technical Report. Risc0. <https://dev.risczero.com/proof-system-in-detail.pdf> Accessed: 2024-09-10.
- [10] Patricia Sandro. 2022. Columbia Ranked No. 2 by U.S. News. But a Math Professor Has His Doubts. *The New York Times* (Sep 2022). <https://www.nytimes.com/2022/09/12/us/columbia-university-us-news-ranking.html> Accessed: October 10, 2024.
- [11] Srinath Setty, Justin Thaler, and Riad Wahby. 2024. Unlocking the Lookup Singularity with Lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*. Springer, 180–209.
- [12] Fatemeh Heidari Soureshjani, Mathias Hall-Andersen, Mohammad Mahdi Jahanara, Jeffrey Kam, Jan Gorzny, and Mohsen Ahmadvand. 2023. Automated Analysis of Halo2 Circuits. *Cryptology ePrint Archive* (2023).
- [13] U.S. Department of Health and Human Services. 2023. Summary of the HIPAA Privacy Rule. <https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html>. Accessed: 2024-10-11.
- [14] U.S. Food and Drug Administration. 2023. CFR - Code of Federal Regulations Title 21. <https://www.accessdata.fda.gov/scripts/cdrh/cfdocs/cfrcfr/cfrsearch.cfm>. Accessed: 2024-10-11.
- [15] U.S. Securities and Exchange Commission. 2023. The Role of the SEC. <https://www.sec.gov/about/what-we-do>. Accessed: 2024-10-11.
- [16] Haodi Wang, Yu Guo, Rongfang Bie, and Xiaohua Jia. 2023. Verifiable Arbitrary Queries with Zero Knowledge Confidentiality in Decentralized Storage. *IEEE Transactions on Information Forensics and Security* (2023).
- [17] Xinxuan Zhang and Yi Deng. 2023. Zero-Knowledge Functional Elementary Databases. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 269–303.